Learnings and Reflection Report

Real Estate Project Feasibility and Financial Appraisal Tool. CE725 Construction Economics and Finance, IIT Bombay. Course Project, Team 8, Topic 2.

Maneesha (23B4223), Project Lead and Finance Modeller. Dinesh Sai (23B1022), Data Lead. Sujay (23B1059), AI and Tool Developer. Surya Theja (23B1062), Validation Engineer. Akshitha (23B0629), Documentation and Reflection Lead.

Prepared 5 November 2026 with AI assistance from the project record; every statement has been checked by the team against that record (AI usage log entry 21). Sections follow the order required in section 7 of the course brief.

How the work was divided. The Project Lead set the method, the conventions and the base case, and took every finance decision. The Data Lead sourced and checked the base-case inputs against published figures. The AI and Tool Developer built the engine, the interface, the tests and the deployment with the AI assistant, and decided what to accept from it. The Validation Engineer owned the benchmark workbook, the manual benchmark and the discrepancy register. The Documentation and Reflection Lead kept the AI usage log, wrote the user manual and compiled this report. Every member read the validation report and the data source list before signing off on this document.

1. Subject learnings

This section records what the team understands about construction economics and finance now that it did not understand at the start of the project. Each point came from a specific result, a specific error or a specific source, and we cite it.

1.1 A headline IRR hides the shape of the cash flow

The base case reports an equity IRR of 47.2% and a project IRR of 34.2%. At the start of the project most of us would have quoted the first figure and stopped. The difference between the two is leverage: half of the land, approval and construction spend is borrowed at 12% a year, which is well below the project's own return, so the equity earns more than the project does. The same leverage produces a peak equity requirement of ₹12.86 crore in month 8, a peak debt of ₹12.08 crore and ₹2.02 crore of interest. A developer who looks only at IRR sees none of that. A lender looks only at the project return and the debt schedule and ignores the equity IRR entirely. We now understand that the two figures answer two different questions, "how good is this deal for the promoter?" and "how good is this project?", and that quoting one without the other is misleading.

Profit on cost showed the same thing from a different side. Pre-tax profit on cost is 32.7% and post-tax 24.5%. Both are respectable margins and neither says anything about time. A project that earns 32.7% on cost over 36 months is a different investment from one that earns it over 60 months, which is why the discounted measures, not the margin, carry the decision.

1.2 Timing of sales matters as much as price

In the tornado chart a 3-month delay in the sales launch costs almost as much NPV as a 10% rise in construction cost, and more than a 10% rise in land cost. Delay costs money twice. Collections arrive later, so they are discounted more heavily, and the loan stays outstanding longer, so interest accrues for more

months. We had learned in class that time value of money matters; we had not appreciated that in a leveraged project the interest effect is of the same order as the discounting effect.

A second timing result surprised us in the other direction. The Claude-generated memo pointed out that a 3-month construction delay *raises* NPV in our model, from ₹7.45 crore to ₹7.82 crore. The memo flagged this as counter-intuitive, and it is, but it is correct given the assumptions: the model has no delay penalty, no cost escalation and no customer compensation, so pushing spend later only defers it. In reality a delay brings escalation, idle overheads, RERA interest to buyers and lost sales. The model's result is a reminder that a DCF only prices the cash flows it is given.

1.3 NPV is the reliable decision measure; IRR is not always

While running a heavy-debt case through the Excel workbook we found two rates at which NPV is zero, −13.8% and 839% a year. The tool reported one and Excel the other. Both were mathematically correct and neither was useful, while NPV at 15% was an unambiguous +₹8.68 crore. The cause was that the equity cash flow changed sign twice: outflows during construction, inflows from collections, then a large bullet repayment of the loan in the final month. We had read about multiple IRRs in a textbook. We had not expected to produce one with inputs that a developer could plausibly enter. The tool now counts the roots in a plausible band, checks that the IRR's position relative to the hurdle rate agrees with the sign of NPV, and tells the user to decide on NPV when it does not.

1.4 NPV does not always fall when the discount rate rises

One of our automated tests asserted that NPV falls as the discount rate rises. It failed on 5 of 40 random input sets. Two patterns break the rule. In a loss-making project every period nets to an outflow, so a higher rate shrinks the present value of the loss and NPV rises towards zero. In a project with inflows first and outflows later, a higher rate shrinks the later outflows more than the earlier inflows. The rule we had learned holds for profitable, conventional cash flows only. The model was right; our assumption was wrong. This changed how we read every sensitivity row: a move in the "wrong" direction is not necessarily a bug.

1.5 Break-even has two meanings

The tool reports two break-even prices. At ₹6,249.5 per sq ft equity NPV is zero; at ₹5,686.5 per sq ft the project merely covers its costs including interest and tax. The 10% gap between them is the return the equity demands at 15%. A developer who reports only the second number is saying the project cannot lose money, which is true, while hiding that it cannot earn its cost of capital either. Both numbers were reproduced by Goal Seek in the workbook, which was the clearest demonstration we had of what "the price at which NPV is zero" actually means.

1.6 Conventions must be stated or two correct calculations disagree

End-of-month discounting, an effective rate converted to 1.1715% a month rather than a nominal 1.25%, interest on the opening balance rather than the closing balance, instalments starting the month after booking, and whether total cost includes interest each change the answer by amounts that would look like errors. Our Excel and Python results matched to fourteen decimal places only because both followed one written list of conventions, now section 7 of the user manual. The hand calculation in section 6 of the validation report matched to the fourth decimal for the same reason. We now understand why a feasibility appraisal always begins with an assumptions page.

1.7 What the market data taught us

Sourcing the base case taught more finance than building the model did.

- **Three prices for the same flat.** For Kalyan on the same day we found an asking price of ₹9,750, a ready reckoner rate of ₹6,039 and a registered transaction rate of ₹7,900 per sq ft. An appraisal that uses the first overstates revenue by a quarter; one that uses the second understates it. Our ₹7,500 is defensible only against the third.
- **Collections are slower than a model likes.** RERA section 13(1) caps the advance before a registered agreement at 10%, and MahaRERA-format schedules spread the balance over slab milestones across 18 to 30 months. Our 6-month instalment pattern is optimistic, and we say so in the manual. The 70% escrow rule in section 4(2)(l)(D) is also why lenders fund the early spend: the developer cannot freely use collections.
- **Approvals are a cost line, not a formality.** Inside Greater Mumbai, MCHI-CREDAI put direct municipal charges at 23% to 32% of project cost. Our ₹3 crore (5.3% of total cost) is plausible only for the periphery. This single input decides whether a Greater Mumbai project is viable at all.
- **The cost of debt sits on a policy rate.** The repo rate was 5.50% from 7 October 2026 and construction finance from a bank or a category-A NBFC was 10% to 14%. The 6 to 8 point spread over the policy rate is the market's view of development risk, and it is why a 2-point rise in the interest rate is a realistic stress, not an extreme one.
- **The hurdle rate is a survey result, not a constant.** The EY/NSE cost of capital survey puts the cost of equity for Indian real estate at 16.4%; a developer survey reported in an MDPI paper gives a median IRR hurdle of 18%. Our 15% is below both. We kept it because it is the rate taught in the course and the sensitivity rows show the result at 16% and 17%, but we now know that 15% is a choice to be defended.

1.8 Debt is a schedule, not a percentage

Before the project "50% debt" was a single number. Modelling it taught us that a construction loan is a sequence of decisions: what spend it funds, when it is drawn, whether interest is paid or capitalised, what share of collections is swept to repayment, and what happens to any balance at the end. The first version of the model funded only construction with debt. Collections began before construction peaked and the sweep repaid the loan almost immediately, so peak debt was ₹0.79 crore and interest ₹0.04 crore. The numbers were arithmetically correct and economically absurd. The corrected model funds land, approvals and construction, and peak debt rose to ₹12.08 crore. Month 8 is instructive: on the hand calculation the interest, the drawdown, the sweep and the repayment cap must be applied in the right order or the balance is wrong, and a bullet repayment at the end is what created the second IRR root.

1.9 Tax timing and the equity view

Paying tax in a single final month rather than spreading it is a simplification, but it showed us how a large late outflow changes the picture. In month 36 the equity cash flow is −₹4.21 crore: the last instalment comes in, the ₹4.62 crore tax goes out. A promoter who has already distributed the profit would be caught short. The equity view, where the tax and the loan repayment are the last two lines, is the one a promoter must plan cash against, and it is different from the project view a lender uses.

2. AI capability assessment

The AI assistant (Claude Code) wrote almost all of the code and the first draft of every document. It was fast and usually right, and it was wrong in specific ways that we caught only because we tested. This section gives the evidence on both sides.

2.1 Where it accelerated the work

- The full cash flow engine, the metrics, the sensitivity module, the Streamlit interface and the Excel workbook generator were produced in one working session on 30 October.
- It proposed the smoothstep S-curve, $3x^2 - 2x^3$, precisely because it can be reproduced in a spreadsheet cell, which is what made a formula-only benchmark workbook possible.
- It wrote 383 automated tests, including identities checked on 200 random input sets and agreement with the numpy_financial library. We would not have written a tenth of these by hand.
- It built the browser test with Playwright, 27 checks that drive the real interface, and later pointed it at the cloud deployment with a one-line change.
- It explained its own errors clearly when asked, and in several cases labelled its own output as insufficient (the manual benchmark, the untested memo path) rather than claiming more than it had done.
- During data collection it found and summarised nineteen sources in an afternoon and built the input-by-input comparison table in the data source list.

2.2 Where it was wrong, with evidence

#	Error	Type	How we found it
1	IRR function returned "None" for the base case	Faulty code: bisection bracket of −99% to +1000% per month had the same NPV sign at both ends	First run of the base case
2	Debt modelled on construction only, giving ₹0.04 crore of interest	Wrong assumption: did not consider that collections start before construction peaks	Reading the output and asking whether the numbers made sense
3	Asserted "NPV falls as the discount rate rises" as a universal test	Wrong finance assumption	Randomised tests failed on 5 of 40 cases
4	Sensitivity table with 15 mostly empty columns	Faulty code: labels per driver type created extra columns	A test on table shape
5	Tornado colours inverted for cost drivers	Faulty logic: coloured by input direction, not metric direction	Looking at a screenshot
6	Template memo contained a number (16.7%) not traceable to the fact sheet	Incomplete design	Its own number-check guard
7	First multiple-IRR detector flagged the base case because of a root at −98% a year	Faulty logic: search band too wide	Running the detector on the base case
8	Server restart command killed itself	Tooling error: kill pattern matched its own shell	Empty test output
9	Wrote that land cost was the third-widest tornado bar in the manual	Fabricated ordering, stated without checking	Re-running the tornado showed sales delay was third
10	Reported the RBI repo rate as 5.25%	Stale data: a search summary from before the 7 October 2026 increase	Opening the RBI press release: 5.50%
11	Quoted a mean IRR hurdle of 18.17% and a range of 10% to 30% from an academic paper	Fabricated statistics: the page gives only a median of 18% and a standard deviation	The Data Lead opened the paper in a browser

#	Error	Type	How we found it
12	Said a circle-rate website labelled its figures as estimates	Fabricated attribution from a search summary	Opening the page
13	Cited page numbers in the Knight Frank report and an RBI press release number without having opened them	Unverified citation	The team asked; the AI then opened both and corrected the page and the release number
14	One input change took 27.7 s in the browser	Inefficient code: sensitivity loops solved two break-even prices per appraisal they did not need	Timing the browser test

Errors 1, 4, 5, 7, 8 and 14 are faulty code or logic. Errors 2 and 3 are wrong finance assumptions, and these are the dangerous kind, because the code ran and produced plausible numbers. Errors 9, 11, 12 and 13 are fabrication: the AI stated something as fact that it had not checked, in the confident register it uses for things it has checked. Error 10 is stale data presented as current. None of the fourteen was caught by reading the code or the prose. All were caught by running the output against something independent: a benchmark, a test, a screenshot, an opened source, or common sense about the numbers.

2.3 How the fabrications looked from the inside

The three data-collection fabrications deserve a note because they are the kind an evaluator could not detect from the finished document. In each case the AI had a search summary rather than the page, and it wrote as if it had the page. The mean of 18.17% was plausible, the right order of magnitude, and wrong. It would have gone into the data source list and been cited in this report if the Data Lead had not opened the paper and found only a median. The lesson we drew is procedural: any number the AI attributes to a source is treated as unverified until a person has seen the source, and the data source list now has a column recording how each source was seen and by whom.

2.4 What the AI did not do

It did not do the manual benchmark on its own authority. It produced a calculator-style first pass on 5 November, labelled it as not satisfying the course requirement, and the Validation Engineer re-performed every line with a calculator before any of it was accepted.

It could not test the Claude-generated memo until the team supplied an API key on 5 November. Until then it flagged the gap in the test report rather than filling it with invented results. When the memo was run, through the AIML API gateway with Claude Sonnet 5.5 and then Claude Opus 5.5, every number in both memos matched the fact sheet. The memos also did two things we had not asked for. The first pointed out that the fact sheet did not state what each scenario changed, so it could not judge their likelihood; we added the scenario definitions. The second noticed that the model rewards a construction delay (section 1.2); we recorded that as a known limitation. In both cases the language model was a useful reviewer of the inputs it was given, and in neither case did it invent a figure.

2.5 Our overall assessment

The AI is a fast producer of plausible first versions and a poor judge of when it is wrong. It shifted the developer's job from writing code to designing checks, and the Data Lead's job from finding figures to verifying them. It saved us weeks on the build and cost us hours on verification, and the trade was worthwhile only because we made the verification non-negotiable. We would use it again for every task on this project, and for none of them without a benchmark.

3. Validation insights

3.1 The layers

The course requires the base case to be recomputed without AI assistance and compared with the tool. We used four layers of mechanical checking and one manual benchmark.

Layer	What it is	Independence from the Python engine	Result
A	Excel workbook of pure formulas, no values copied from Python, recalculated by LibreOffice Calc	Different calculation engine, same written conventions	Every summary metric and every monthly column agrees to better than 1e−11
B	Three further input cases (downside, heavy debt, no debt) run through copies of the workbook	Tests the formulas on inputs other than the one they were built around	Agreement on every figure except the heavy-debt IRR, see 3.3
C	Decimal-arithmetic recomputation written from the assumptions list	Independent re-implementation, exact arithmetic	Matches the engine to 1.5e−14
D	383 automated tests: hand-worked examples, identities on 200 random input sets, numpy_financial	Internal consistency and third-party agreement	All pass; one test was wrong and was corrected (section 1.4)
Manual	Months 1, 8 and 36 and the 36-row NPV sum by calculator; Goal Seek for both break-even prices, by the Validation Engineer	A person following the written conventions	All figures agree to the fourth decimal; NPV 7.4514 against 7.4515

3.2 What agreement proved, and what it did not

Layers A and C were written by the same AI from the same assumptions list as the engine. Their agreement with the engine to fourteen decimal places proves that the three implementations share one reading of the conventions. It does not prove that the reading is right. A misunderstanding in the assumptions, such as charging interest on the closing balance, would propagate to all three and they would still agree. This is why the course insists on a manual benchmark, and why we treat the hand calculation, not the workbook, as the validation.

The hand calculation was done against the same conventions, so it too shares the reading. What it adds is a person who had to apply each convention consciously, in order, with a calculator. The Validation Engineer reports that month 8 was where the conventions bit: the S-curve share for that month, the instalment on the previous month's bookings, interest on the opening balance, the drawdown, and the repayment capped at the outstanding balance each had to be done in sequence, and the order is where a mistake would hide. The only differences from the tool were in the fourth decimal, from rounding the monthly discount factor.

3.3 The disagreements taught more than the agreement

The validation report, section 7, lists ten discrepancies. The ones that mattered:

- **IRR returned nothing for a clearly profitable base case** (discrepancy 1). The AI's bisection bracket had the same NPV sign at both ends because of the final tax outflow. Found on the first run; fixed with a grid scan. The lesson is that a root-finder's bracket is an assumption about the cash flow shape.
- **Excel's own IRR() failed** on the downside case until given a starting guess, while the tool gave 13.3% (discrepancy 3). A spreadsheet is not automatically more trustworthy than code; its IRR is a Newton iteration with a default guess that can miss.

- **Two valid IRRs** in the heavy-debt case (discrepancy 4). This was the one genuine disagreement between engines, and it was not an arithmetic error but a property of the finance. Validation found a limitation of the method, not of the code, and the tool changed to detect it.
- **The test was wrong, the model was right** (discrepancy 8). Five random cases violated an assertion we had written from classroom intuition. We corrected the test and kept the model.
- **A performance problem** (discrepancy 9) was found only by timing the browser, not by any unit test. 27.7 s per input change made the app unusable; the cause was two break-even solves per appraisal inside the sensitivity loops.

Two things stand out. First, every discrepancy that involved finance rather than code was found by a check on inputs the base case does not exercise: a heavy-debt case, a loss-making random case, a downside case. The base case is well-behaved by construction and would never have revealed them. Second, the discrepancy register is more useful to a reader than the agreement table, because it says what can go wrong and how one would notice.

3.4 What we would validate next

With more time the Validation Engineer would hand-check one month of the heavy-debt case, where the bullet repayment occurs, because that is the case where the engines disagreed. We would also have a practising developer or a lender's credit analyst enter a project they know and tell us where the output looks wrong; the course offers bonus credit for this and we did not manage it before the deadline.

4. Practical relevance

4.1 Who would use it as it stands

- **A developer or promoter** could use the tool for a first screening of a residential project: does the deal clear the hurdle rate, how much equity does it need and when, and which three inputs decide the answer. The tornado chart is the screen a promoter would look at first.
- **A lender's credit analyst** could use the debt schedule, the peak debt figure and the stress scenarios to size a facility and to see how fast a sweep repays it. The project IRR and the unlevered view are the numbers they would quote internally.
- **A feasibility consultant** could use it to produce the first draft of an appraisal and to document the assumptions, because every convention is written down and the Excel workbook reproduces the result.
- **A site engineer or contractor** would not use it. The tool models the developer's cash flow, not the contractor's; a contractor's feasibility is about bid price, retention, mobilisation advance and running bills, none of which are here. The construction S-curve is the only shared element.

4.2 What would have to change for real use

Change	Why
Construction-linked payment plans in place of booking plus equal instalments	Real MahaRERA schedules collect on slab milestones over 18 to 30 months; our 6-month pattern collects too fast
Cost escalation and price inflation by period	A 27-month construction window sees material and labour escalation; the model is in constant prices
GST, stamp duty, registration and the RERA 70% escrow	These change both the cash collected and when the developer can use it
Interest capitalisation and a moratorium	Most construction finance capitalises interest during construction; our model pays it monthly from equity

Change	Why
Phased launches, multiple unit types, unsold inventory and a tail of sales after completion	A single price and a single window is a screening simplification
A delay penalty: idle overheads, RERA interest to buyers, escalation	Without it the model rewards construction delay (section 1.2)
Approval cost as an itemised premium schedule for the specific municipality	The single input decides viability inside Greater Mumbai and is our weakest figure
Probabilistic treatment of price and absorption	Price and sales pace are the two drivers with the widest real-world range; one-at-a-time sensitivity does not show their joint effect
Audit trail and versioning of inputs	A lender needs to know which assumptions were used when a figure was quoted

4.3 The memo

The investment memo was the feature a non-technical reader valued most and the one we trust least. The template version is safe: it fills fixed sentences with the computed numbers and cannot invent one. The Claude version reads better and, in our tests, was a genuinely useful reviewer of the inputs, but it is only as good as the person who reads it, and it sends the fact sheet to an external service. A real deployment would keep the number check, log every memo with its fact sheet, and require a named person to sign the memo before it leaves the firm.

5. Individual reflections (half a page each)

> **Note.** These five reflections were drafted with AI assistance on 5 November 2026 from the project record and each member has read and verified their own. The AI's role in drafting them is recorded in AI usage log entry 21.

Maneesha, Project Lead and Finance Modeller. My role was to own the economic logic: the choice of a monthly discounted cash flow at the equity level, the conventions in section 7 of the manual, and the base case. The thing I understand now that I did not at the start is that IRR is not a property of a project but of a cash flow pattern. When we ran a heavy-debt case, the tool said −14% and Excel said 839%, and both were right, because a large bullet repayment makes the equity cash flow change sign twice. Until then I had treated IRR as the number a developer quotes; now I would quote NPV at a stated hurdle rate and treat IRR as a cross-check. The second learning was how much leverage flatters equity IRR: 47% equity IRR against 34% project IRR is the loan working, not the project, and a lender would only look at the second figure. What I would do differently is decide the discount rate from evidence before building rather than after: the EY/NSE survey puts the real estate cost of equity at 16.4% and we built on 15%, which is defensible but was a guess at the time. The decision I am least sure of is keeping the base case unchanged after the data check; it preserved the benchmark but it means the tool's default is a little kinder than the market. We worked in two blocks: the build on 30 October, when the engine, interface, workbook and tests were produced and the first discrepancies found, and 5 November, when the data check, the live memo, the manual benchmark and the deployment were done. My job in both was to decide, not to type. The decision where I overrode the AI's data check was the discount rate and the tax rate: it flagged 16.4% and 25.17% as the better-supported values, and I kept 15% and 25% so that the benchmark, the validation report and the screenshots stayed consistent, with the sensitivity rows showing what the sourced values would do.

Dinesh Sai, Data Lead. My job was to find out whether the base case numbers were real. The honest answer is that for the first week they were not: they were round, plausible figures the AI proposed and we accepted, and no document said where they came from. Fixing that meant choosing a market, the peripheral Mumbai Metropolitan Region, and checking every input against published sources. The most useful thing I learned is the difference between an asking price, a ready reckoner rate and a registered transaction rate: for Kalyan these were ₹9,750, ₹6,039 and ₹7,900 per sq ft on the same day, and our ₹7,500 only looks right against the last of them. The second thing was that search summaries lie by omission. The first repo rate the AI reported was 5.25%; the RBI had raised it to 5.50% a month earlier, and only opening the RBI release caught it. I opened the MDPI paper myself when the AI's tools were blocked and found that the mean IRR hurdle it had quoted was not on the page; only the median was. After that I asked for a "how seen" column in the source table so that nobody, including us, could mistake a summary for a document. For a real project I would want the actual municipal premium schedule for the plot, a construction-linked payment plan from a comparable launch, and a lender term sheet, because the three inputs we had to label optimistic (instalments, marketing, approvals) are exactly the ones that a published average cannot settle. The source check took one afternoon for nineteen sources, most of it spent opening documents rather than finding them. The hardest were the MCHI-CREDAI approval-cost paper, whose website blocked the AI's fetch and which I had to read from a saved copy, and a Business Standard article on Mumbai premiums that sat behind a paywall; we withdrew that one rather than cite a figure none of us had read.

Sujay, AI and Tool Developer. I was responsible for the build and for deciding what to accept from the AI. The pattern that worked was never accepting code on the strength of it running. Every function was accepted only when something independent agreed with it: the Excel workbook, the numpy_financial library, a property test over 200 random input sets, or a screenshot. That caught the errors that mattered: the IRR function returned nothing for the base case because its bracket was wrong; the loan was almost unused because it funded construction only; the sensitivity table had fifteen columns; and one input change took 28 seconds because the sensitivity loop was solving two break-even prices it did not need. None of these would have been found by reading the code, and all of them were found by testing it. Deploying to Streamlit Community Cloud taught one more: the browser test timed out against the live site because the cloud wraps the app in an iframe, and the fix was to point the test at the inner URL, not to change the app. The suggestion I rejected outright was the AI's first multiple-IRR detector, which flagged our own base case as having more than one IRR because it found a second root at −98% a year. A warning that fires on the benchmark project is worse than no warning, because users would learn to ignore it; the detector was rewritten to search only a plausible band and to cross-check the IRR against the sign of NPV. What I now believe about AI-assisted coding is that the AI is fast at producing a plausible version and poor at knowing when it is wrong; the developer's job moves from writing the code to designing the checks that would expose it.

Surya Theja, Validation Engineer. My role was the manual benchmark, and the course is right to insist on it. The Excel workbook and the Python engine agreed to fourteen decimal places, which sounds like proof until you remember both were written by the same AI from the same assumptions; agreement between them only proves they share a reading. Redoing months 1, 8 and 36 with a calculator was the first time a person had followed the written conventions independently. Month 8 was the instructive one: getting the S-curve share, the instalment on the previous month's booking, the interest on the opening balance and the repayment cap in the right order takes about twenty minutes, and the only differences from the tool were in the fourth decimal, from my own rounding. The NPV sum of 36 present values came

to 7.4514 against the tool's 7.4515 for the same reason. Goal Seek reproduced both break-even prices. What the discrepancies in section 7 of the validation report taught me is that most of them were not arithmetic: they were the AI's test being wrong about finance (NPV does not always fall when the rate rises) or Excel's own IRR failing without a starting guess. With more time I would hand-check the month of the bullet repayment in the heavy-debt case, because that is the one place the two engines disagreed. The whole check, three months, the 36-row NPV sum and the two Goal Seeks, was done in one sitting on 5 November; the NPV sum was the longest part, because every row needs its own discount factor and a slip in one row is invisible until the total disagrees.

Akshitha, Documentation and Reflection Lead. I owned the AI usage log, the user manual and this report. The hardest part of the log was keeping the Decision column honest. It is easy to write Accept under every entry; the entries worth reading are the Modify and Reject ones, and the ones where the AI's output was wrong and we say so: the stale repo rate, the mean hurdle rate that was not in the paper, the tornado colours that were inverted, the test assumptions that were false. A log with no rejections would tell an evaluator we had not been checking. For the manual, the discipline was length. The first version was eight pages and read as a tour of the screen; the submitted four pages keep only what a user needs to act: inputs, how to read each metric, the conventions, where the numbers came from and what is not modelled. Writing for a director who will never open the tool also shaped the memo: the number check that flags any figure not in the fact sheet exists because a reader of the memo cannot verify it themselves. Compiling this report showed me that the project's real output is not the tool but the record around it: the conventions, the discrepancy register, the source list and the log are what would let someone else trust a number. No one outside the team has used the tool yet. Watching teammates during the browser test, the first thing a new user does is change the price and look for the NPV to move, and only later asks what the number means; so the change I would make first is a quarter-page quick start at the top of the manual, three steps from one input to the tornado chart, before the section that explains each metric.

6. Ethics and limitations

6.1 Data reliability

The base case was proposed as round, plausible figures and only later checked against published sources. After the check, every input has a source or is declared an assumption in the data source list, and three inputs are labelled optimistic: the 6-month instalment pattern, the 2% marketing cost, and the approval cost if the site is inside Greater Mumbai. We chose not to change the base case after the check, because changing it would have invalidated the Excel benchmark, the validation report and the screenshots. That decision is disclosed in the manual and the data source list. The consequence is that the tool's default project is a little kinder than the market, and a user must enter their own inputs for a real site.

The sources themselves have limits. Reports from brokers and consultants summarise the market they sell into; a registered-price index covers only the deals that reached registration; a 2017 approval-cost paper is at 2017 rates. We cite the year and the kind of source for each figure so that a reader can weigh it. One source was withdrawn because it sat behind a paywall and could not be read; we did not keep a figure we could not see.

6.2 Over-reliance on AI

The AI produced working code within minutes, which creates pressure to trust it. Fourteen of its errors are listed in section 2, and several produced plausible-looking numbers or plausible-looking citations. The

safeguard was not reading the output more carefully but checking it against something the AI did not write: a calculator, an opened source, a screenshot, a library written by someone else. We also note that the AI generated the spreadsheet we validated against, so the independence of that layer is partial, and we say so in the validation report rather than presenting the workbook as an independent check.

The subtler risk was in the documents. The AI drafted the manual, the validation report and this report, and a draft that reads well invites acceptance. Every claim in those drafts that cites a number or a source has been traced to the record by a team member, and the ones that could not be traced were removed. We treat "the AI wrote it and it sounded right" as no evidence at all.

6.3 Accountability for decisions based on AI-generated numbers

A number produced by this tool and quoted in a decision belongs to the person quoting it, not to the tool, the AI or the team that built it. The memo tab says this on screen. The number-check guard on the Claude memo reduces but does not remove the risk of a fabricated figure, because it only checks that each number appears in the fact sheet, not that the sentence around it is sound. A developer who takes this tool's output to a bank without re-entering their own inputs and reading the assumptions page would be acting on our assumptions, not theirs. We built the assumptions page, the conventions list and the data source list so that no one can reasonably claim not to have known what the numbers rest on.

6.4 Confidentiality

The Claude memo mode sends the fact sheet, which contains every input and every computed metric, to an external API. The tool says so on screen before the button is pressed. For a confidential project the template mode, which runs offline and sends nothing, is the only appropriate choice. The team's API key is stored outside the repository and is never committed.

6.5 Limitations of the tool

The model is in constant prices, pays interest monthly from equity, collects on a fixed booking-plus-instalment pattern, has one unit type and one price, pays tax in a single month, and has no delay penalty. Sensitivity is one-at-a-time; there is no probabilistic analysis. The base case was checked against published figures for the peripheral Mumbai Metropolitan Region and nowhere else. The tool was tested on Linux and Chrome and on Streamlit Community Cloud; no practising developer or lender has yet used it. These limitations are stated in the user manual so that a user can judge whether the tool fits their case.